Name: Aaditya Surve
Batch: T22
Roll no.: 2401119
Branch: AI&DS

EXPERIMENT NO. 3

Deploying Multi-Service Applications Using Docker Compose

LO4: Containerization, orchestrate and deploy applications using Docker, Compose, Kubernetes and Cloud infrastructure tools.

1. Aim

To understand the concept of multi-container orchestration and to deploy a multi-service application consisting of a Nginx web server and a MySQL database using Docker Compose, covering service definition, environment configuration, networking, and volume persistence.

2. Software Requirements

- Windows 10/11 (64-bit)
- Docker Desktop (includes Docker Compose)
- WSL2 (Windows Subsystem for Linux)
- PowerShell / Command Prompt
- Text Editor (Notepad / VS Code)
- Active Internet Connection
- Web Browser

3. Theory

3.1 What is Docker Compose?

Docker Compose is a tool provided by Docker for defining and running multi-container applications. Instead of manually creating and linking multiple containers using separate 'docker run' commands, Compose allows all services, networks, and volumes to be declared in a single YAML file (docker-compose.yml). A single command can then be used to start, stop, or rebuild the entire application stack.

3.2 Why Multi-Service Applications?

Modern applications are rarely single-process systems. A typical web application requires a frontend/web server, a back-end application server, and a database, each of which may need to scale, restart, or be updated independently. Running each component as its own container, while connecting them through a shared Compose-managed network, mirrors real-world microservice architecture and simplifies development, testing, and deployment.

3.3 Key Concepts in Docker Compose

- **Services** — each entry under 'services:' represents one container (e.g., web, database).
- **Image** — the base image used to create the container for a service.
- **Ports** — mapping of host machine ports to container ports (HOST:CONTAINER).
- **Environment Variables** — used to configure containers at runtime (e.g., database credentials).

- **Volumes** — persistent storage that survives container removal, used here for MySQL data.
- **Networks** — Compose automatically creates a shared network so services can communicate using their service names as hostnames.

3.4 Structure of docker-compose.yml

The Compose file follows YAML syntax and is organized under top-level keys: 'version' (Compose file format version), 'services' (list of containers to run), and 'volumes' (named volumes for persistent storage). Each service can specify its image, container name, ports, environment variables, and volume mounts.

3.5 Advantages of Docker Compose

- Single command to start/stop an entire multi-container application.
- Declarative configuration — the whole stack is defined in one version-controlled file.
- Automatic internal networking between services using service names.
- Simplified environment variable and volume management.
- Easier to reproduce identical environments across machines.

4. Setup and Configuration

The following steps describe how to set up the project directory and Compose file.

1. Open PowerShell / Command Prompt.
2. Create a new directory for the project:

```
C:\Users\203>mkdir compose-demo
```

3. Navigate into the newly created directory:

```
C:\Users\203>cd compose-demo
```

4. Open Notepad (or any text editor) to create the Compose file:

```
C:\Users\203\compose-demo>notepad docker-compose.yml
```

5. Paste the following YAML configuration into the file:

```yaml
version: "3.8"

services:
  web:
    image: nginx
    container_name: webserver
    ports:
      - "8080:80"

  database:
    image: mysql:8.0
    container_name: mysqlserver
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: studentdb
      MYSQL_USER: student
      MYSQL_PASSWORD: student123
    ports:
      - "3306:3306"
    volumes:
      - mysql-data:/var/lib/mysql

volumes:
  mysql-data:
```

6. Save the file. In Notepad's 'Save As' dialog, set 'Save as type' to 'All Files' and name it exactly 'docker-compose.yml', saving it inside the compose-demo directory.
7. Verify the file was created correctly by listing directory contents:

```
C:\Users\203\compose-demo>dir
 Volume in drive C has no label.
 Volume Serial Number is 72A1-7F7F

 Directory of C:\Users\203\compose-demo

29-07-2026  13:48    <DIR>          .
29-07-2026  13:46    <DIR>          ..
29-07-2026  13:50               348 docker-compose.yml
               1 File(s)            348 bytes
               2 Dir(s)  166,185,181,184 bytes free
```

5. Commands Used

Command	Description
`docker compose up`	Start all services defined in the Compose file
`docker compose up -d`	Start all services in detached (background) mode
`docker compose ps`	Show status of running services
`docker compose logs`	View logs of all services
`docker compose stop`	Stop running services without removing them

Command	Description
`docker compose start`	Restart previously stopped services
`docker compose restart`	Restart containers for all services
`docker compose down`	Stop and remove containers, networks created by up
`docker compose down -v`	Stop and remove containers, networks, and volumes
`docker compose images`	Show images used by the created containers
`docker compose config`	Validate and view the resolved Compose file

6. Procedure

The steps below were carried out in sequence, all from within the compose-demo directory in PowerShell / Command Prompt, to deploy and manage the multi-service application:

#	Command	Explanation	Expected Output
1	`mkdir compose-demo`	Create project directory.	Directory created.
2	`cd compose-demo`	Move into project directory.	Prompt path changes to compose-demo.
3	`notepad docker-compose.yml`	Create and edit the Compose file.	Notepad opens a new file.
4	`(paste YAML, save as 'All Files')`	Define web (nginx) and database (mysql) services.	File saved as docker-compose.yml.
5	`dir`	Verify docker-compose.yml exists in the folder.	docker-compose.yml listed.
6	`docker compose config`	Validate YAML syntax before starting.	Resolved configuration printed, no errors.
7	`docker compose up -d`	Build/pull images and start both services in background.	Both containers created and started.
8	`docker compose ps`	Confirm both containers are running.	webserver and mysqlserver shown as 'running'.
9	`Browser -> localhost:8080`	Access the Nginx service via mapped port.	Nginx welcome page loads.
10	`docker compose logs`	View combined logs of both services.	Logs from web and database services shown.
11	`docker exec -it mysqlserver mysql -u student -p`	Connect to MySQL inside the database container.	Prompted for password; MySQL shell opens.
12	`SHOW DATABASES;`	Verify studentdb was created via environment variables.	studentdb listed among databases.
13	`docker compose stop`	Stop both running services.	Containers stop, status 'Exited'.
14	`docker compose start`	Restart the stopped services.	Containers return to 'running' state.
15	`docker compose down`	Stop and remove containers and network (volume persists).	Containers and network removed.
16	`docker compose up -d`	Bring the stack back up to verify data persistence.	studentdb still present due to named volume.

#	Command	Explanation	Expected Output
17	`docker compose down -v`	Clean up completely, including the volume.	Containers, network, and volume all removed.

7. Screenshots

Screenshots of the stages of execution are shown below.

```
C:\Users\203\compose-demo>docker compose config
time="2026-07-29T13:55:57+05:30" level=warning msg="C:\\Users\\203\\compose-demo\\docker-compose.yml: the attribute `ver
sion` is obsolete, it will be ignored, please remove it to avoid potential confusion"
name: compose-demo
services:
  database:
    container_name: mysqlserver
    environment:
      MYSQL_DATABASE: studentdb
      MYSQL_PASSWORD: student123
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_USER: student
    image: mysql:8.0
    networks:
      default: null
    ports:
      - mode: ingress
        target: 3306
        published: "3306"
        protocol: tcp
    volumes:
      - type: volume
        source: mysql-data
        target: /var/lib/mysql
        volume: {}
  web:
    container_name: webserver
    image: nginx
    networks:
```

Figure 1(a): docker compose config — validating and printing the resolved configuration

```
    networks:
      default: null
    ports:
      - mode: ingress
        target: 80
        published: "8080"
        protocol: tcp
networks:
  default:
    name: compose-demo_default
volumes:
  mysql-data:
    name: compose-demo_mysql-data
```

Figure 1(b): docker compose config — remaining output (networks and volumes)

```
C:\Users\203\compose-demo>docker compose up -d
time="2026-07-29T13:56:00+05:30" level=warning msg="C:\\Users\\203\\compose-demo\\docker-compose.yml: the attribute `ver
sion` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 4/4
 ✔Network compose-demo_default    Created                                                    0.1s
 ✔Volume compose-demo_mysql-data  Created                                                    0.0s
 ✔Container mysqlserver           Started                                                    0.6s
 ✔Container webserver             Started                                                    0.6s
```

Figure 2: docker compose up -d — network, volume and both containers created and started

```
C:\Users\203\compose-demo>docker compose ps
time="2026-07-29T13:59:43+05:30" level=warning msg="C:\\Users\\203\\compose-demo\\docker-compose.yml: the attribute `version` is obsolete, it will be ignore
d, please remove it to avoid potential confusion"
NAME          IMAGE       COMMAND                  SERVICE    CREATED         STATUS         PORTS
mysqlserver   mysql:8.0   "docker-entrypoint.s…"   database   3 minutes ago   Up 3 minutes   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
webserver     nginx       "/docker-entrypoint.…"   web        3 minutes ago   Up 3 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

Figure 3: docker compose ps — webserver and mysqlserver running

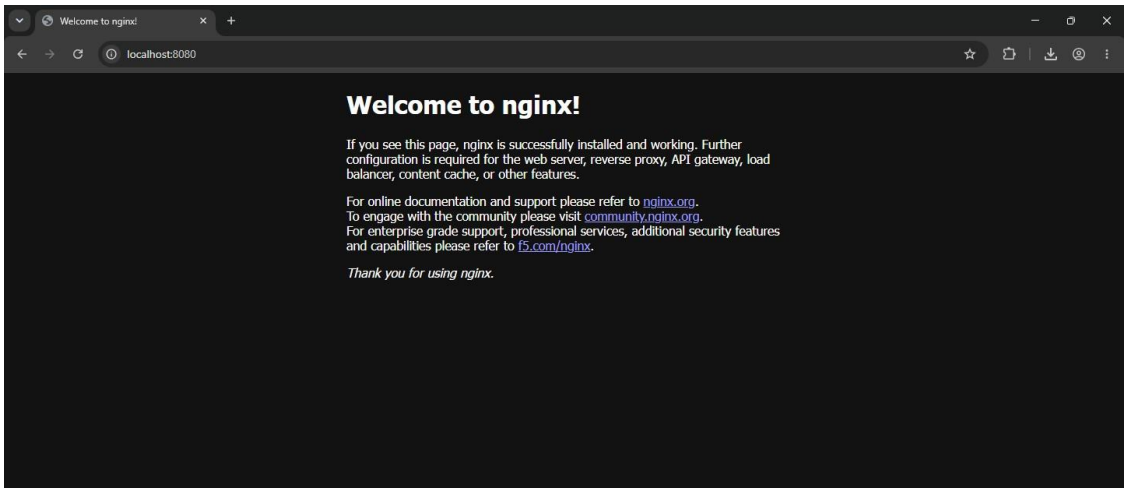


Figure 4: Nginx welcome page served at localhost:8080

```
C:\Users\203\compose-demo>docker exec -it mysqlserver mysql -u student -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| performance_schema |
| studentdb          |
+--------------------+
3 rows in set (0.00 sec)
```

Figure 5: Connecting to MySQL in the mysqlserver container and listing databases (studentdb present)

```
C:\Users\203\compose-demo>docker compose down -v
time="2026-07-29T14:08:07+05:30" level=warning msg="C:\\Users\\203\\compose-demo\\docker-compose.yml: the attribute `version` is obsolete, it will be ignore
d, please remove it to avoid potential confusion"
[+] down 4/4
 ✔Container mysqlserver          Removed                                                                                                          1.3s
 ✔Container webserver            Removed                                                                                                          0.4s
 ✔Volume compose-demo_mysql-data Removed                                                                                                          0.0s
 ✔Network compose-demo_default   Removed                                                                                                          0.2s
```

Figure 6: docker compose down -v — containers, volume and network removed

8. Observation

The experiment demonstrated that Docker Compose offers a more streamlined and efficient method for managing multi-container applications than manually executing separate docker run commands for each service. Both the Nginx web server and the MySQL database were deployed, networked, and configured through a single YAML configuration file, making the setup process simple and consistent. The Nginx service was successfully accessed via the mapped host port, while the MySQL container was initialized with the required database, username, and password using environment variables. Additionally, the use of a named volume ensured that database data remained intact even after the containers were stopped and removed with docker compose down. When the docker compose down -v command was executed, both the containers and the associated volumes were removed, resulting in a complete cleanup of the application's resources.

9. Result

A multi-container application comprising an Nginx web server and a MySQL database was successfully deployed using Docker Compose. Both services were launched simultaneously with a single command, and their functionality was verified to ensure proper operation. The complete lifecycle of the application—including starting, stopping, restarting, and removing the services—was managed collectively, demonstrating the effectiveness of Docker Compose in orchestrating and managing interconnected containers.

10. Conclusion

This experiment offered hands-on experience in managing multi-container applications using Docker Compose. It demonstrated how a single YAML configuration file can be used to define and manage multiple interconnected services, including their images, port mappings, environment variables, and persistent storage. Unlike manually handling individual containers, Docker Compose simplifies deployment and management while providing a consistent and version-controlled approach to configuring an entire application stack. The experiment also highlighted the role of volumes in ensuring data persistence and showed how Docker Compose automatically creates networks that enable seamless communication between services. Overall, it emphasized the efficiency of deploying, scaling, and removing related services as a unified application, which is an essential practice in modern DevOps and microservices-based environments.